



AtliQ Hardware – Advanced SQL Analytics

Contents

- Introduction
- ER Diagram
- Project Tasks:
 - a. Sales Analytics
 - b. Finance Analytics
 - c. Supply Chain Analytics

Introduction

AtliQ Hardware is an imaginary computer peripheral manufacturing company that sells products worldwide.

Customers :

- **Brick and Mortar** - physical stores such as Croma ,Best Buy .
- **E-commerce** - online websites like amazon, Flipkart.

Sales Channels :

- **Retailer** (Brick and Mortar, E-commerce)
- **Direct** (AtliQ e store, AtliQ exclusive)
- **Distributor** (Neptune)

The database consists of **1.4 M - 2 M** records providing real-world scale for analysis.

Tables

The dataset includes multiple fact and dimension tables.

Relationships between Sales, Customer, Product, and Market entities are modeled using an ER diagram for efficient querying and analysis.

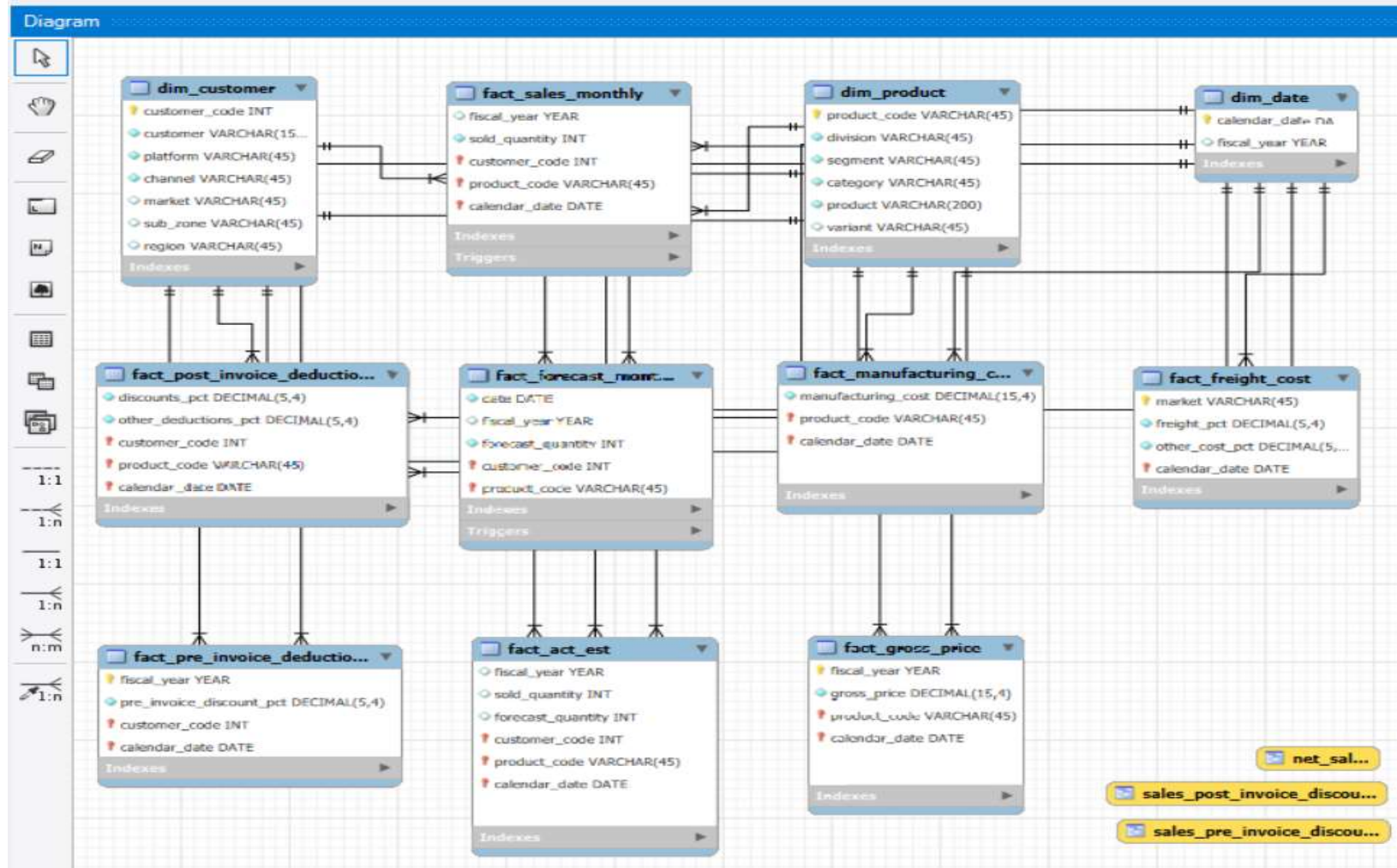
The screenshot displays a SQL IDE interface. On the left, the 'SCHEMAS' pane shows a database named 'gdb0041' with the following structure:

- Tables**
 - dim_customer
 - dim_date
 - dim_product
 - fact_act_est
 - fact_forecast_monthly
 - fact_freight_cost
 - fact_gross_price
 - fact_manufacturing_cost
 - fact_post_invoice_deductions
 - fact_pre_invoice_deductions
 - fact_sales_monthly
- Views**
 - net_sales
 - sales_post_invoice_discount
 - sales_pre_invoice_discount
- Stored Procedures**
 - get_forecast_accuracy
 - get_market_badge
 - get_monthly_gross_sales_for_customers
 - net top n customers by net sales

On the right, the 'Query 1' pane shows the following SQL code:

```
16
17  -- Number of Records in Fact tables
18  • SELECT count(*) FROM fact_sales_monthly;    -- '1425706' = 1.4 million records
19  • SELECT count(*) FROM fact_forecast_monthly; -- '1885941' = 1.8 million
20  • SELECT count(*) FROM fact_post_invoice_deductions; -- '2063076' = 2 Million 63 thousand 76 rows
21  • SELECT count(*) FROM fact_freight_cost;    -- 135
22  • SELECT count(*) FROM fact_gross_price;    -- 1182
23  • SELECT count(*) FROM fact_manufacturing_cost; -- 1182
24  • SELECT count(*) FROM fact_pre_invoice_deductions; -- 1045
25
26  -- Number of Records in Dimension tables
27  • SELECT count(*) FROM dim_customer;        -- 209
28  • SELECT count(*) FROM dim_product;        -- 397
29  • SELECT count(*) FROM dim_date;           -- 87    derived table for fiscal_year
30
31
32
33  --
```

ER Diagram



Task 1 : Generate **Monthly Sales Report** for a given Customer .

```
163  /* TASK 4 Use Stored Procedure to AUTOMATE the Process of generating monthly sales of any given customer */
164  •  SELECT s.date,
165         SUM(ROUND(s.sold_quantity * g.gross_price,2)) as monthly_sales
166  FROM fact_sales_monthly s
167  JOIN fact_gross_price g
168  ON g.fiscal_year = get_fiscal_year(s.date) AND
169     g.product_code = s.product_code
170  WHERE customer_code = 90002002
171  GROUP BY date;
172
173
```

<		
Result Grid		
Filter Rows:		
Export:  Wrap Cell Content: 		
	date	monthly_sales
▶	2017-09-01	122407.57
	2017-10-01	162687.56
	2017-12-01	245673.84
	2018-01-01	127574.73
	2018-02-01	144799.54
	2018-04-01	130643.92

Name: get_monthly_gross_sales_for_customers

DDL:

```

1 CREATE DEFINER='root'@'localhost' PROCEDURE `get_monthly_gross_sales_for_customers`(
2     in_customer_codes TEXT)
3 BEGIN
4     SELECT s.date,
5           SUM(ROUND(s.sold_quantity * g.gross_price,2)) as monthly_sales
6     FROM fact_sales_monthly s
7     JOIN fact_gross_price g
8     ON g.fiscal_year = get_fiscal_year(s.date) AND
9        g.product_code = s.product_code
10    WHERE FIND_IN_SET(s.customer_code, in_customer_codes) > 0
11    GROUP BY date;
12 END

```

Outcomes :

- Joined fact_sales_monthly with pricing tables to calculate **monthly gross sales**.
- Automated the report generation process using **Stored Procedures** for any given customer.

Navigator: SCHEMAS

Filter objects

gdb0041

- Tables
 - dim_customer
 - dim_product
 - fact_forecast_monthly
 - fact_freight_cost
 - fact_gross_price
 - fact_manufacturing_cost
 - fact_post_invoice_deductions
 - fact_pre_invoice_deductions
 - fact_sales_monthly
- Views
- Stored Procedures
 - get_monthly_gross_sales_for_custom
- Functions
 - f() get_fiscal_quarter
 - f() get_fiscal_year

Administration Schemas

Information

Procedure: get_monthly_gross_sales_for_customers

SQL Prj Queries

fact_gross_price dim_customer get_fiscal_quarter - Routine get_fiscal_year - Routine

Limit to 1000 rows

```

1 call gdb0041.get_monthly_gross_sales_for_customers('90002008,90002016');
2

```

Result Grid

date	monthly_sales
2017-10-01	438917.81
2017-11-01	575530.58
2017-12-01	617049.84
2018-02-01	358033.19
2018-03-01	384656.75

Task 2 : Generate Gross Sales Report using User Defined Functions .Display Month, Product name, Variant, Sold quantity, Gross price per item, Gross price total.



Functions : get_fiscal_year & get_fiscal_quarter

The screenshot shows a SQL IDE interface with a sidebar on the left containing a 'SCHEMAS' tree. Under 'gdb0041', there are 'Tables' and 'Functions'. The 'Functions' folder is expanded, showing 'f() get_fiscal_quarter' and 'f() get_fiscal_year'. The main editor area is titled 'get_fiscal_year - Routine'. It contains the following SQL code:

```
1 CREATE DEFINER='root'@'localhost' FUNCTION `get_fiscal_year`(calendar_date DATE) RETURNS int
2 DETERMINISTIC
3 BEGIN
4     DECLARE fiscal_year INT;
5     SET fiscal_year = YEAR(DATE_ADD(calendar_date,INTERVAL 4 MONTH));
6     RETURN fiscal_year;
7 END
```

The screenshot shows the same SQL IDE interface, but now the 'get_fiscal_quarter - Routine' is selected in the main editor. The SQL code is as follows:

```
1 CREATE DEFINER='root'@'localhost' FUNCTION `get_fiscal_quarter`(calendar_date date) RETURNS char(2) CHARSET utf8mb4
2 DETERMINISTIC
3 BEGIN
4     DECLARE m TINYINT ;
5     DECLARE qtr CHAR(2) ;
6     SET m = month(calendar_date);
7
8     CASE
9         WHEN m IN(9,10,11) THEN
10         SET qtr = "Q1";
11         WHEN m IN(12,1,2) THEN
12         SET qtr = "Q2";
13         WHEN m IN(3,4,5) THEN
14         SET qtr = "Q3";
15         ELSE
16         SET qtr = "Q4";
17     END CASE;
18     RETURN qtr;
19 END
```

SQL Prj Queries* x fact_gross_price dim_customer get_fiscal_quarter get_fiscal_quarter - Routine get_fiscal_year - Routine

Limit to 1000 rows

```

114
115 • SELECT s.date,s.product_code,
116         p.product,p.variant,s.sold_quantity,
117         g.gross_price,
118         ROUND(g.gross_price * s.sold_quantity,2) as gross_price_total
119 FROM fact_sales_monthly s
120 JOIN dim_product p
121 ON p.product_code = s.product_code
122 JOIN fact_gross_price g
123 ON g.product_code = s.product_code and g.fiscal_year = get_fiscal_year(s.date) -- 2 common columns
124 WHERE customer_code = 90002002 and
125        get_fiscal_year(date) = 2021 and
126        get_fiscal_quarter(date) = "Q4"
127 ORDER BY date
128 LIMIT 1000000;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	date	product_code	product	variant	sold_quantity	gross_price	gross_price_total
▶	2021-06-01	A0118150101	AQ Dracula HDD – 3.5 Inch SATA 6 Gb/s 5400 R...	Standard	205	19.0573	3906.75
	2021-06-01	A0118150102	AQ Dracula HDD – 3.5 Inch SATA 6 Gb/s 5400 R...	Plus	78	21.4565	1673.61
	2021-06-01	A0118150103	AQ Dracula HDD – 3.5 Inch SATA 6 Gb/s 5400 R...	Premium	48	21.7795	1045.42
	2021-06-01	A0118150104	AQ Dracula HDD – 3.5 Inch SATA 6 Gb/s 5400 R...	Premium Plus	126	22.9729	2894.59
	2021-06-01	A0219150201	AQ WereWolf NAS Internal Hard Drive HDD – 8....	Standard	40	23.6987	947.95
	2021-06-01	A0219150202	AQ WereWolf NAS Internal Hard Drive HDD – 8....	Plus	102	24.7312	2522.58
	2021-06-01	A0220150203	AQ WereWolf NAS Internal Hard Drive HDD – 8....	Premium	31	23.6154	732.08
	2021-06-01	A0320150301	AQ Zion Saga	Standard	91	23.7223	2158.73
	2021-06-01	A0321150302	AQ Zion Saga	Plus	70	27.1027	1897.19
	2021-06-01	A0321150303	AQ Zion Saga	Premium	145	28.0059	4060.86
	2021-06-01	A0418150103	AQ Mforce Gen X	Standard 3	108	19.5235	2108.54
	2021-06-01	A0418150104	AQ Mforce Gen X	Plus 1	76	19.9239	1514.22
	2021-06-01	A0418150105	AQ Mforce Gen X	Plus 2	84	20.0766	1686.43
	2021-06-01	A0418150106	AQ Mforce Gen X	Plus 3	114	19.9365	2272.76

Outcomes :

- Created **User-Defined Functions**: get_fiscal_year and get_fiscal_quarter.
- Displayed month, product, variant, sold quantity, gross price per item, and total gross price.
- Exported results to **CSV** for further Excel analysis.

Task 3 : Stored Procedure for Market Badge



Limit to 1000 rows

Result Grid

Filter Rows:

Export:

Stored Procedure – get_market_badge



Navigator: SQL Prj Queries* fact_gross_price dim_customer **get_market_badge - Routine** get_market_badge get_market_badge

SCHEMAS

Filter objects

- dim_product
- fact_forecast_monthly
- fact_freight_cost
- fact_gross_price
- fact_manufacturing_cost
- fact_post_invoice_deductions
- fact_pre_invoice_deductions
- fact_sales_monthly
- Views
- Stored Procedures
 - get_market_badge**
 - get_monthly_gross_sales_for_custorm
- Functions
 - f() get_fiscal_quarter
 - f() get_fiscal_year
- my_movies
- random_tables
- revision

Administration Schemas

Information: Unable to retrieve node description.

Name: get_market_badge

DDL:

```
3  IN in_fiscal_year year,
4  OUT out_badge varchar(45))
5  BEGIN
6      DECLARE qty int default 0;
7      # set default market to be India
8      if in_market = "" then
9          set in_market = "India";
10     end if;
11     # retrieve total qty for a given market_qty+ financial_year
12     SELECT sum(sold_quantity) into qty
13     FROM fact_sales_monthly s
14     JOIN dim_customer c
15     ON s.customer_code = c.customer_code
16     WHERE get_fiscal_year(s.date) = in_fiscal_year and c.market = in_market
17     GROUP BY c.market;
18
19     # determine market badge
20     if qty > 5000000 then
21         set out_badge = "Gold";
22     else
23         set out_badge = "Silver";
24     end if;
25     END
```

SQL Prj Queries* fact_gross_price dim_customer get_market_badge - Routine get_market_badge get_market_badge get_market_badge - Routine get_market_badge - Routine

Name: get_market_badge The name of the routine is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:

```

1 CREATE DEFINER='root'@'localhost' PROCEDURE `get_market_badge`(
2   IN in_market varchar(45),
3   IN in_fiscal_year year,
4   OUT out_badge varchar(45))
5 BEGIN
6   -- Call stored procedure gdb0041.get_market_badge
7   -- Enter values for parameters of your procedure and click <Execute> to create an SQL editor
8   -- and run the call:
9
10  in_market USA [IN] varchar(45)
11  in_fiscal_year 2020 [IN] year
12  out_badge [OUT] varchar(45)
13
14  Execute Cancel
15
16
17 GROUP BY c.market;
18
19 # determine market badge
20 if qty > 5000000 then
21   set out_badge = "Gold";
22 else
23   set out_badge = "Silver";
24 and if:

```

SQL Prj Queries* fact_gross_price dim_customer get_market_badge - Routine get_market_badge

Limit to 1000 rows

```

1 set @out_badge = '0';
2 call gdb0041.get_market_badge('USA', 2020, @out_badge);
3 select @out_badge;
4

```

Result Grid

@out_badge
Silver

Filter Rows: Export: Wrap Cell Content: I A

Outcomes :

- Developed a **Stored Procedure** get_market_badge to classify markets as : **Gold** - Total sold quantity > 5M and **Silver** – Otherwise
- Combined fact_sales_monthly and dim_customer to calculate sales by market.

Task 4 Generate Report for Top Markets, Products and Customers for given financial year using Views



Projects / AtliQ Data Analytics / Add epic / AFA-19

Top markets, products, customers for a given financial year

Attach Add a child issue Link issue ...

276693

Description

As a product owner, I want a report for top markets, products, customers by net sales for a given financial year so that I can have a holistic view of our financial performance and can take appropriate actions to address any potential issues.

We will probably write stored proc for this as we will need this report going forward as well.

1. Report for **top markets**.

Rank	Market	Net Sales (in millions)
1	India	210.67
2	USA	132.05
3	South Korea	64.01

2. Report for **top products**.

Rank	Product	Net Sales
------	---------	-----------



View : Sales_pre_invoice_discount



Name: new_view

The name of the view is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:



```
1 • CREATE VIEW `sales_pre_invoice_discount` AS
2     SELECT
3         s.date,
4         s.fiscal_year,
5         s.customer_code,
6         c.market,
7         s.product_code,
8         p.product,
9         p.variant,
10        s.sold_quantity,
11        g.gross_price as gross_price_per_item,
12        ROUND(s.sold_quantity*g.gross_price,2) as gross_price_total,
13        pre.pre_invoice_discount_pct
14    FROM fact_sales_monthly s
15    JOIN dim_customer c
16        ON s.customer_code = c.customer_code
17    JOIN dim_product p
18        ON s.product_code=p.product_code
19    JOIN fact_gross_price g
20        ON g.fiscal_year=s.fiscal_year
21        AND g.product_code=s.product_code
22    JOIN fact_pre_invoice_deductions as pre
23        ON pre.customer_code = s.customer_code AND
24        pre.fiscal_year=s.fiscal_year
25
```



Name: sales_pre_invoice_discount

The name of the view is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:



```
3     DEFINER = 'root'@'localhost'
4     SQL SECURITY DEFINER
5     VIEW `sales_pre_invoice_discount` AS
6     SELECT
7         `s`.`date` AS `date`,
8         `s`.`fiscal_year` AS `fiscal_year`,
9         `s`.`customer_code` AS `customer_code`,
10        `c`.`market` AS `market`,
11        `s`.`product_code` AS `product_code`,
12        `p`.`product` AS `product`,
13        `p`.`variant` AS `variant`,
14        `s`.`sold_quantity` AS `sold_quantity`,
15        `g`.`gross_price` AS `gross_price_per_item`,
16        ROUND(((`s`.`sold_quantity` * `g`.`gross_price`),
17            2) AS `gross_price_total`,
18        `pre`.`pre_invoice_discount_pct` AS `pre_invoice_discount_pct`
19    FROM
20        (((`fact_sales_monthly` `s`
21        JOIN `dim_customer` `c` ON (((`s`.`customer_code` = `c`.`customer_code`)))
22        JOIN `dim_product` `p` ON (((`s`.`product_code` = `p`.`product_code`)))
23        JOIN `fact_gross_price` `g` ON (((`g`.`fiscal_year` = `s`.`fiscal_year`
24            AND (`g`.`product_code` = `s`.`product_code`))))
25        JOIN `fact_pre_invoice_deductions` `pre` ON (((`pre`.`customer_code` = `s`.`customer_code`)
26            AND (`pre`.`fiscal_year` = `s`.`fiscal_year`))))
```

Views : Sales_post_invoice_discount and net_sales

Name: sales_post_invoice_discount

The name of the view is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:

```

1 • CREATE
2     ALGORITHM = UNDEFINED
3     DEFINER = `root`@`localhost`
4     SQL SECURITY DEFINER
5     VIEW `sales_post_invoice_discount` AS
6     SELECT
7         `s`.`date` AS `date`,
8         `s`.`fiscal_year` AS `fiscal_year`,
9         `s`.`customer_code` AS `customer_code`,
10        `s`.`market` AS `market`,
11        `s`.`product_code` AS `product_code`,
12        `s`.`product` AS `product`,
13        `s`.`variant` AS `variant`,
14        `s`.`sold_quantity` AS `sold_quantity`,
15        `s`.`gross_price_total` AS `gross_price_total`,
16        `s`.`pre_invoice_discount_pct` AS `pre_invoice_discount_pct`,
17        (`s`.`gross_price_total` - (`s`.`pre_invoice_discount_pct` * `s`.`gross_price_total`)) AS `net_invoice_sales`,
18        (`po`.`discounts_pct` + `po`.`other_deductions_pct`) AS `post_invoice_discount_pct`
19     FROM
20     (`sales_pre_invoice_discount` `s`
21     JOIN `fact_post_invoice_deductions` `po` ON (((`po`.`customer_code` = `s`.`customer_code`)
22         AND (`po`.`product_code` = `s`.`product_code`)
23         AND (`po`.`date` = `s`.`date`))))

```

Name: net_sales

The name of the view is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:

```

1 • CREATE
2     ALGORITHM = UNDEFINED
3     DEFINER = `root`@`localhost`
4     SQL SECURITY DEFINER
5     VIEW `net_sales` AS
6     SELECT
7         `sales_post_invoice_discount`.`date` AS `date`,
8         `sales_post_invoice_discount`.`fiscal_year` AS `fiscal_year`,
9         `sales_post_invoice_discount`.`customer_code` AS `customer_code`,
10        `sales_post_invoice_discount`.`market` AS `market`,
11        `sales_post_invoice_discount`.`product_code` AS `product_code`,
12        `sales_post_invoice_discount`.`product` AS `product`,
13        `sales_post_invoice_discount`.`variant` AS `variant`,
14        `sales_post_invoice_discount`.`sold_quantity` AS `sold_quantity`,
15        `sales_post_invoice_discount`.`gross_price_total` AS `gross_price_total`,
16        `sales_post_invoice_discount`.`pre_invoice_discount_pct` AS `pre_invoice_discount_pct`,
17        `sales_post_invoice_discount`.`net_invoice_sales` AS `net_invoice_sales`,
18        `sales_post_invoice_discount`.`post_invoice_discount_pct` AS `post_invoice_discount_pct`,
19        (`sales_post_invoice_discount`.`net_invoice_sales` * (1 - `sales_post_invoice_discount`.`post_invoice_discount_pct`)) AS `net_sales`
20     FROM
21     `sales_post_invoice_discount`

```

Stored Procedure to get top n customers by Net_sales

Name:

DDL:

```
1 CREATE DEFINER='root'@'localhost' PROCEDURE `get_top_n_customers_by_net_sales`(  
2   in_market varchar(45),  
3   in_fiscal_year INT,  
4   in_top_n INT  
5 )  
6 BEGIN  
7   select customer,  
8     round(sum(net_sales)/1000000,2) as net_sales_mln  
9   from net_sales n  
10  join dim_customer c  
11  on n.customer_code = c.customer_code  
12  where fiscal_year = in_fiscal_year  
13  group by c.customer  
14  order by net_sales_mln desc  
15  limit in_top_n;  
16 END
```

Navigator: net_sales get_top_n_markets - Routine get_top_n_customers_by_net_s...

SCHEMAS

Filter objects

- fact_freight_cost
- fact_gross_price
- fact_manufacturing_cost
- fact_post_invoice_deductions
- fact_pre_invoice_deductions
- fact_sales_monthly
- Views
 - net_sales
 - sales_post_invoice_discount
 - sales_pre_invoice_discount
- Stored Procedures
 - get_market_badge
 - get_monthly_gross_sales_for_custo
 - get_top_n_customers_by_net_sales
 - get_top_n_markets
- Functions
 - f() get_fiscal_quarter
 - f() get_fiscal_year

Administration Schemas

Information

Schema: gdb0041

SQL Prj Queries* x net_sales get_top_n_markets - Routine get_top_n_customers_by_net_s...

Limit to 1000 rows

```
368 -- Create a Stored Procedure to store this result  
369 -- Now lets go for Top Customers  
370 select customer,  
371       round(sum(net_sales)/1000000,2) as net_sales_mln  
372 from net_sales n  
373 join dim_customer c
```

Call stored procedure gdb0041.get_top_n_customers_by_n...

Enter values for parameters of your procedure and click <Execute> to create an SQL editor and run the call:

in_market [IN] varchar(45)

in_fiscal_year [IN] INT

in_top_n [IN] INT

Execute Cancel

Fetch rows:

Navigator

SQL Prj Queries* net_sales get_top_n_markets - Routine get_top_n_customers_by_net_s... get_top_n_custome

SCHEMAS

Filter objects

- fact_freight_cost
- fact_gross_price
- fact_manufacturing_cost
- fact_post_invoice_deductions
- fact_pre_invoice_deductions
- fact_sales_monthly
- Views
 - net_sales
 - sales_post_invoice_discount
 - sales_pre_invoice_discount
- Stored Procedures
 - get_market_badge
 - get_monthly_gross_sales_for_custom
 - get_top_n_customers_by_net_sales
 - get_top_n_markets
- Functions
 - f() get_fiscal_quarter
 - f() net_fiscal_year

Administration Schemas

Limit to 1000 rows

```
1 • call gdb0041.get_top_n_customers_by_net_sales('India', 2021, 3);
2
```

Result Grid

customer	net_sales_mln
Amazon	109.03
Atliq Exclusive	79.92
Atliq e Store	70.31

Result 1 x

Stored Procedure :Top_n_products

The screenshot shows the SQL IDE with the DDL for the stored procedure `top_n_products_by_net_sales`. The procedure is defined as follows:

```

1 CREATE DEFINER='root'@'localhost' PROCEDURE `top_n_products_by_net_sales`(
2   in_fiscal_year int,
3   in_top_n int
4 )
5 BEGIN
6   select
7     product,
8     round(sum(net_sales)/1000000,2) as net_sales_min
9   from net_sales
10  where fiscal_year = in_fiscal_year
11  group by product
12  order by net_sales_min desc
13  limit in_top_n;
14 END

```

The screenshot shows the SQL IDE with the execution of the stored procedure `top_n_products_by_net_sales`. The execution command is:

```

1 call gdb0041.top_n_products_by_net_sales(2021, 5);
2

```

The result grid shows the following data:

product	net_sales_min
AQ BZ Allin1	33.75
AQ Qwerty	27.84
AQ Trigger	26.95
AQ Gen Y	23.58
AQ Maxima	22.32

Outcomes :

- Determined the **top performing products, and customers** based on Net Sales. Net Sales are the final revenue figure after deducting pre- and post-invoice discounts.
- Used multiple **Views** (`sales_pre_invoice_discount`, `sales_post_invoice_discount`, `net_sales`) to layer financial calculations and simplify the final reporting queries.

Task 5 : Net Sales % in Global Market Share

SQL Prj Queries* x expenses get_top_n_markets - Routine

Limit to 1000 rows

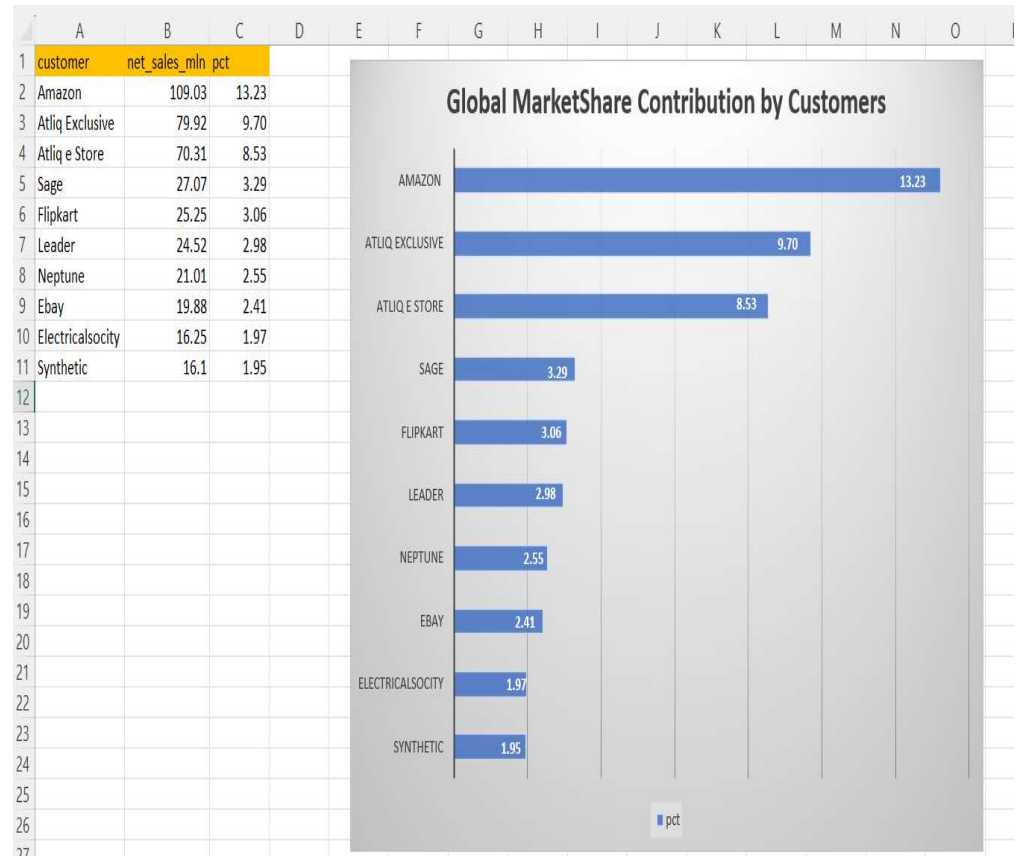
```

432
433 /* TASK 7 To generate a Bar Chart for FY=2021 for top 10 Markets by % NET Sales.
434 Report can be generated in excel. To calculate the percentage contribution of each
435 Market using window functions */
436 with cte1 as (
437     SELECT customer,
438            round(sum(net_sales)/1000000,2) as net_sales_mln
439      from net_sales s
440     join dim_customer c
441      on s.customer_code = c.customer_code
442      where s.fiscal_year = 2021
443      group by customer )
444 select * ,
445        net_sales_mln*100/sum(net_sales_mln) over() as pct
446   from cte1
447  order by net_sales_mln desc
448  limit 10;

```

Result Grid

customer	net_sales_mln	pct
Amazon	109.03	13.233402
Atliq Exclusive	79.92	9.700206
Atliq e Store	70.31	8.533803
Sage	27.07	3.285593
Flipkart	25.25	3.064692
Leader	24.52	2.976089



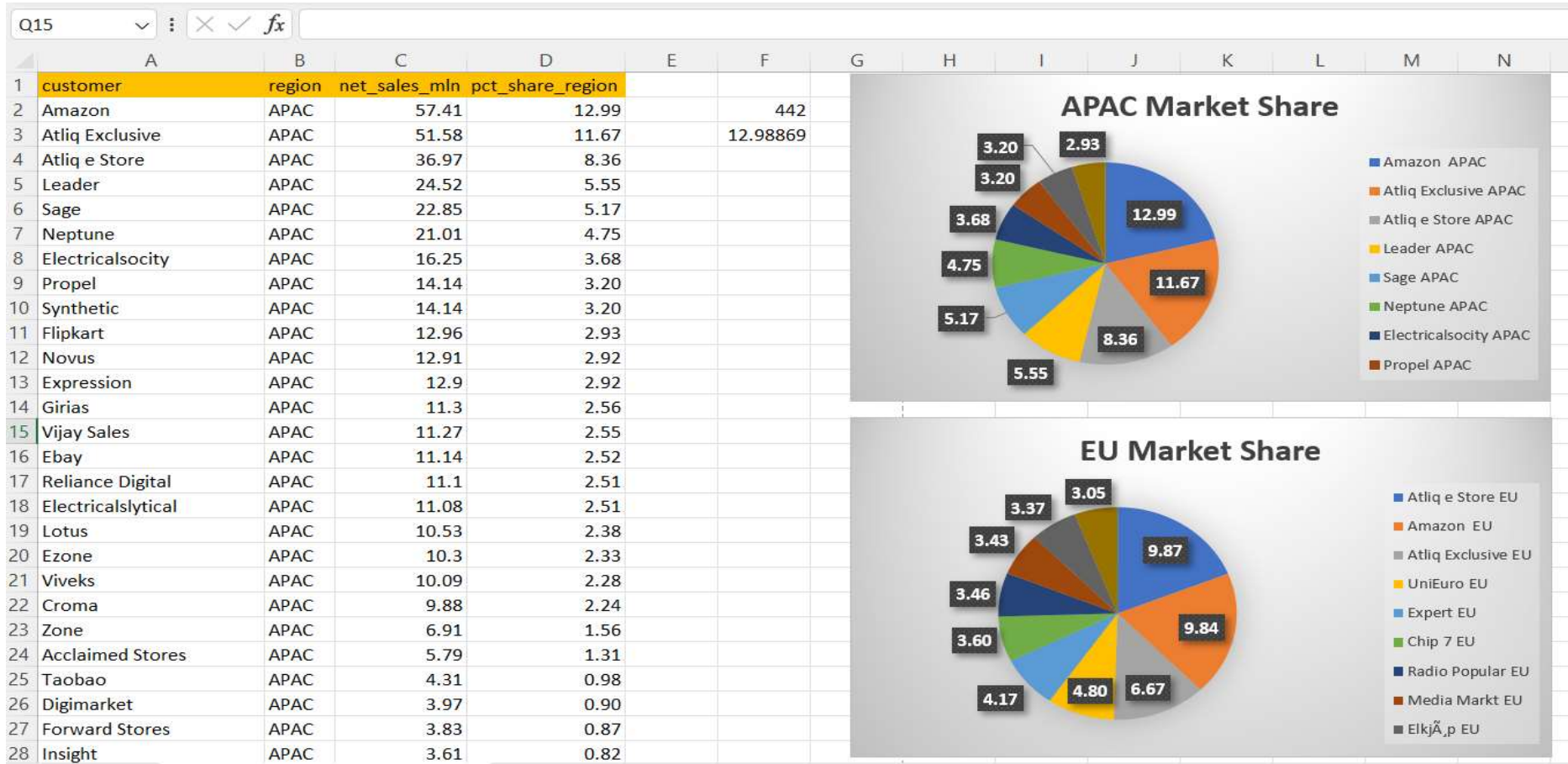
Outcomes :

- Identified the **top customers** by calculating their percentage contribution to the total global net sales.
- Used a **Common Table Expression (CTE)** and **Window Functions** to compute the percentage contribution of each customer to the overall sum.

Task 6 : Generate Region wise Net Sales%

```
475      /* TASK 8 Generate Net Sales % for Regions */
476  • with cte1 as (
477      SELECT  c.customer,c.region,
478              round(sum(net_sales)/1000000,2) as net_sales_mln
479      from net_sales s
480      join dim_customer c
481      on s.customer_code = c.customer_code
482      where s.fiscal_year = 2021
483      group by c.customer,c.region )
484  select *,
485          net_sales_mln* 100/sum(net_sales_mln) over (partition by region) as pct_share_region
486  from cte1
487  order by region,net_sales_mln desc;
```

Result Grid				
Filter Rows:		Export:		Wrap Cell Content:
	customer	region	net_sales_mln	pct_share_region
▶	Amazon	APAC	57.41	12.988688
	Atliq Exclusive	APAC	51.58	11.669683
	Atliq e Store	APAC	36.97	8.364253
	Leader	APAC	24.52	5.547511
	Sage	APAC	22.85	5.169683
	Neptune	APAC	21.01	4.753394
	Electricalsodity	APAC	16.25	3.676471
	Propel	APAC	14.14	3.199095
	Svnthetic	APAC	14.14	3.199095



Outcomes :

- Calculated the individual customer's **market share** within a specific region (e.g., APAC, EU).
- Applied a **Window Function** with a PARTITION BY region clause to calculate net sales percentage relative to the regional total.

Task 7: Generate a report of top n products in each division by their sold quantity

Get top n products in each division by their quantity sold

Attach Add a child issue Link issue

Description

Write a stored proc for getting TOP n products in each division by their quantity sold in a given financial year. For example below would be the result for FY=2021,

Division	Product	Total Quantity
N & S	AQ Pen Drive DRC	2034569
N & S	AQ Digit SSD	1240149
N & S	AQ Clx1	1238683
P & A	AQ Gamers Ms	2477098
P & A	AQ Maxima Ms	2461991
P & A	AQ Master wireless x1 Ms	2448784
PC	AQ Digit	135092
PC	AQ Gen Y	135031
PC	AQ Elite	134431

```

533  /* TASK 9 Write a Stored Procedure to get Top N products in each division by their quantity sold
534  in a given financial year */
535  with cte1 as
536      (select
537          p.division,
538          p.product,
539          sum(sold_quantity) as total_qty
540      from fact_sales_monthly s
541      join dim_product p
542      on p.product_code=s.product_code
543      where fiscal_year=2021
544      group by p.division,p.product),
545      cte2 as (select *,
546          dense_rank() over (partition by division order by total_qty desc) as drnk
547      from cte1)
548      select * from cte2 where drnk<=3;
549

```

Result Grid Filter Rows: Export: Wrap Cell Content: [IA](#)

	division	product	total_qty	drnk
▶	N & S	AQ Pen Drive DRC	2034569	1
	N & S	AQ Digit SSD	1240149	2
	N & S	AQ Clx1	1238683	3
	P & A	AQ Gamers Ms	2477098	1
	P & A	AQ Maxima Ms	2461991	2
	P & A	AQ Master wireless x1 Ms	2448784	3
	PC	AQ Digit	135092	1
	PC	AQ Gen Y	135031	2

SQL Prj Queries* expenses new_procedure - Routine x

Name: new_procedure The name of the routine is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:

```

1 CREATE PROCEDURE `get_top_n_products_per_division_by_qty_sold` (
2     in_fiscal_year INT,
3     in_top_n INT)
4 BEGIN
5     with cte1 as
6     (select
7         p.division,
8         p.product,
9         sum(sold_quantity) as total_qty
10        from fact_sales_monthly s
11        join dim_product p
12        on p.product_code=s.product_code
13        where fiscal_year= in_fiscal_year
14        group by p.division,p.product),
15     cte2 as (select *,
16         dense_rank() over (partition by division order by total_qty desc) as drnk
17        from cte1)
18     select * from cte2 where drnk<=in_top_n ;
19 END

```

Navigator

SCHEMAS

Filter objects

- gdb0041
 - Tables
 - Views
 - Stored Procedures
 - get_market_badge
 - get_monthly_gross_sales_for_customers
 - get_top_n_customers_by_net_sales
 - get_top_n_markets
 - get_top_n_products_per_division_by_qty
 - top_n_products_by_net_sales
 - Functions
 - my_movies
 - random_tables
 - revision
 - sys

Administration Schemas Information

Schema: gdb0041

SQL Prj Queries* expenses get_top_n_products_per_divisio... get_top_n_products_per_divisio... x

Limit to 1000 rows

1 • call gdb0041.get_top_n_products_per_division_by_qty_sold(2021, 5);

2

Result Grid Filter Rows: Export: Wrap Cell Content: IA

division	product	total_qty	drnk
N & S	AQ Pen Drive DRC	2034569	1
N & S	AQ Digit SSD	1240149	2
N & S	AQ Clx1	1238683	3
N & S	AQ Neuer SSD	1225985	4
N & S	AQ Clx2	1201025	5
P & A	AQ Gamers Ms	2477098	1
P & A	AQ Maxima Ms	2461991	2
P & A	AQ Master wireless x1 Ms	2448784	3
P & A	AQ Master wired x1 Ms	2447468	4
P & A	AQ Lite Ms	2443425	5
PC	AQ Digit	135092	1
PC	AQ Gen Y	135031	2
PC	AQ Elite	134431	3
PC	AQ Gen X	134264	4
PC	AQ Velocity	101757	5

Outcomes :

- Identified the **top-performing products** ranked by sold quantity within their respective divisions (e.g., N&S, P&A, PC).
- Used the **DENSE_RANK()** window function partitioned by Division to accurately rank and filter the top N products in each group.

Task 8 : To generate **forecast accuracy** report for given fiscal year to trace the accuracy of the forecast for these customers.

The report should contain following fields:

- 1.Customer Code, Name, Market
- 2.Total Sold Quantity
- 3.Total Forecast Quantity
4. Net Error
- 5.Absolute Error
- 6.Forecast Accuracy%

Query 1 SQL Prj Queries* x sql_tasks

Limit to 2000 rows

```

631 • drop table if exists fact_act_est; /* ===== CREATE A HELPER TABLE */
632 • create table fact_act_est
633 ( select s.date as date,
634         s.fiscal_year as fiscal_year,
635         s.product_code as product_code,
636         s.customer_code as customer_code,
637         s.sold_quantity as sold_quantity,
638         f.forecast_quantity as forecast_quantity
639     from fact_sales_monthly s
640     left join fact_forecast_monthly f
641     using (date, customer_code, product_code)
642 )
643 union
644 ( select f.date as date,
645         f.fiscal_year as fiscal_year,
646         f.product_code as product_code,
647         f.customer_code as customer_code,
648         s.sold_quantity as sold_quantity,
649         f.forecast_quantity as forecast_quantity
650     from fact_forecast_monthly f
651     left join fact_sales_monthly s
652     using (date, customer_code, product_code)
653 )
  
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: I A

	date	fiscal_year	product_code	customer_code	sold_quantity	forecast_quantity
▶	2017-09-01	2018	A0118150101	70002017	51	18
	2017-09-01	2018	A0118150101	70002018	77	11
	2017-09-01	2018	A0118150101	70003181	17	9
	2017-09-01	2018	A0118150101	70003182	6	6
	2017-09-01	2018	A0118150101	70006157	5	5

Query 1

SQL Prj Queries* x sql_tasks

Limit to 2000 rows

```

726
727
728 • with forecast_err_table as (SELECT
729     s.customer_code,
730     sum(s.sold_quantity) as total_sold_qty,
731     sum(s.forecast_quantity) as total_forecast_qty,
732     sum((forecast_quantity - sold_quantity)) as net_err,
733     sum((forecast_quantity - sold_quantity))*100/sum(forecast_quantity) as net_err_pct,
734     sum(abs(forecast_quantity-sold_quantity)) as abs_err,
735     sum(abs(forecast_quantity-sold_quantity))*100/sum(forecast_quantity) as abs_err_pct
736 from gdb0041.fact_act_est s
737 where s.fiscal_year = 2021
738 group by customer_code)
739 select
740     e.* ,
741     c.customer,
742     c.market,
743     if(abs_err_pct > 100,0,100-abs_err_pct) as forecast_accuracy
744 from forecast_err_table e
745 join dim_customer c
746 using(customer_code)
747 order by forecast_accuracy desc;
748

```

Result Grid | Filter Rows: | Exports: | Wrap Cell Content:

	customer_code	total_sold_qty	total_forecast_qty	net_err	net_err_pct	abs_err	abs_err_pct	customer	market	forecast_accuracy
▶	90025209	13178	15428	2065	13.3848	8051	52.1843	Electricalsbea Stores	Columbia	47.8157
	90013120	109547	133532	23944	17.9313	70378	52.7050	Coolblue	Italy	47.2950
	70010048	119439	142010	22547	15.8771	75645	53.2674	Atliq e Store	Bangladesh	46.7326
	90023027	236189	279962	43760	15.6307	149274	53.3194	Costco	Canada	46.6806
	70027208	33713	47321	13342	28.1947	25398	53.6717	Atliq e Store	Brazil	46.3283

The screenshot shows a SQL IDE interface with a schema explorer on the left and a SQL editor in the center. The schema explorer shows a database named 'gdb0041' with various tables and views. The SQL editor displays the DDL for a stored procedure named 'get_forecast_accuracy'. The procedure is created by 'root' at 'localhost' and takes an integer parameter 'in_fiscal_year'. The procedure body includes a CTE named 'forecast_err_table' which selects data from 'fact_sales_monthly' and 'fact_forecast_monthly' tables, joined by 'dim_customer'. The CTE calculates 'total_sold_qty', 'total_forecast_qty', 'net_err', 'abs_err', and 'forecast_accuracy'. The main query selects from 'forecast_err_table' and orders the results by 'forecast_accuracy' in descending order.

A dialog box is shown over the SQL editor, prompting the user to enter values for parameters of the procedure. The 'in_fiscal_year' parameter is set to 2020. The dialog box has 'Execute' and 'Cancel' buttons.

The screenshot shows the same SQL IDE interface, but now the stored procedure 'get_forecast_accuracy' is being executed. The SQL editor shows the call statement: 'call gdb0041.get_forecast_accuracy(2020);'. The 'Result Grid' is displayed at the bottom, showing the output of the procedure. The grid has columns for 'customer_code', 'total_sold_qty', 'total_forecast_qty', 'net_err', 'net_err_pct', 'abs_err', 'abs_err_pct', 'customer', 'market', and 'forecast_accr'.

customer_code	total_sold_qty	total_forecast_qty	net_err	net_err_pct	abs_err	abs_err_pct	customer	market	forecast_accr
90020101	1796	2727	469	17.1984	1187	43.5277	Euronics	Austria	56.4723
90020100	1942	3000	552	18.4000	1312	43.7333	Nova	Austria	56.2667
90020099	1843	2961	580	19.5880	1304	44.0392	Integration Stores	Austria	55.9608
90020097	1889	2778	484	17.4226	1232	44.3485	Atlas Stores	Austria	55.6515
90025209	1811	3312	910	27.4758	1512	45.6522	Electricalsbea Stores	Columbia	54.3478
90020102	2025	2866	471	16.4341	1347	46.9993	Finac-Darty	Austria	53.0007
90020098	1880	2830	582	20.5654	1340	47.3498	Electricalsquipo Stores	Austria	52.6502
90019200	4093	5973	1310	21.9320	3140	52.5699	Sorefoz	Sweden	47.4301
90019201	3708	5461	1264	23.1459	2964	54.2758	Expert	Sweden	45.7242
90019202	3956	5356	969	18.0919	2933	54.7610	Argos (Sainsbury's)	Sweden	45.2390
70020104	3027	2922	-238	-8.1451	1660	56.8104	Atliq e Store	Austria	43.1896
90010046	55532	58644	2971	5.0662	33471	57.0749	Amazon	Banglad...	42.9251
70006158	136991	155044	17949	11.5767	88781	57.2618	Atliq e Store	Philippines	42.7382
90020103	2865	2845	-178	-6.2566	1650	57.9965	Amazon	Austria	42.0035
90023030	127854	140248	12306	8.7745	82458	58.7944	Amazon	Canada	41.2056
70008170	178182	192586	14342	7.4471	113620	58.9970	Atliq e Store	Australia	41.0030

Outcomes :

- **Measured the accuracy** of the company's sales forecasts by comparing Total Sold Quantity (Actual) against Total Forecast Quantity.
- **Created a helper table** using CTE by UNION and joining fact_sales_monthly and fact_forecast_monthly and **calculated key metrics** like Net Error, Absolute Error, and Forecast Accuracy %.

Thank you